

Analytics Studio

Technical Requirements Document (TRD)

Version 1.0 (Final MVP)

Project: Analytics Studio

Internship: ReadyNest – Week 6 Final Project

Architecture: Modular • In-Memory • Fresh Build

Development Style: Prompt-driven (Antigravity) with manual verification after every completed module.

Deadline: Friday, 10:00 PM

1. Purpose

This document defines the complete technical architecture, engineering standards, module contracts, API specifications, coding conventions, validation rules, and implementation guidelines for Analytics Studio.

Unlike the PRD, this document focuses entirely on **how the system will be built**, not what business problem it solves.

2. System Overview

Analytics Studio follows a modular pipeline architecture.

Every module performs one responsibility only.

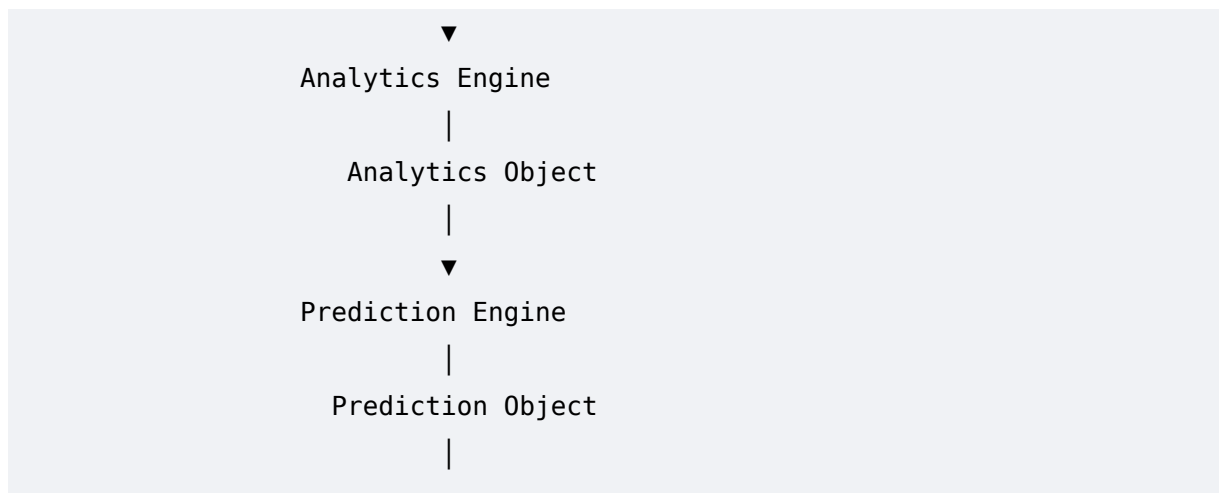
Every module returns structured domain objects.

No module performs presentation logic.

No module communicates using anonymous dictionaries.

Overall Architecture

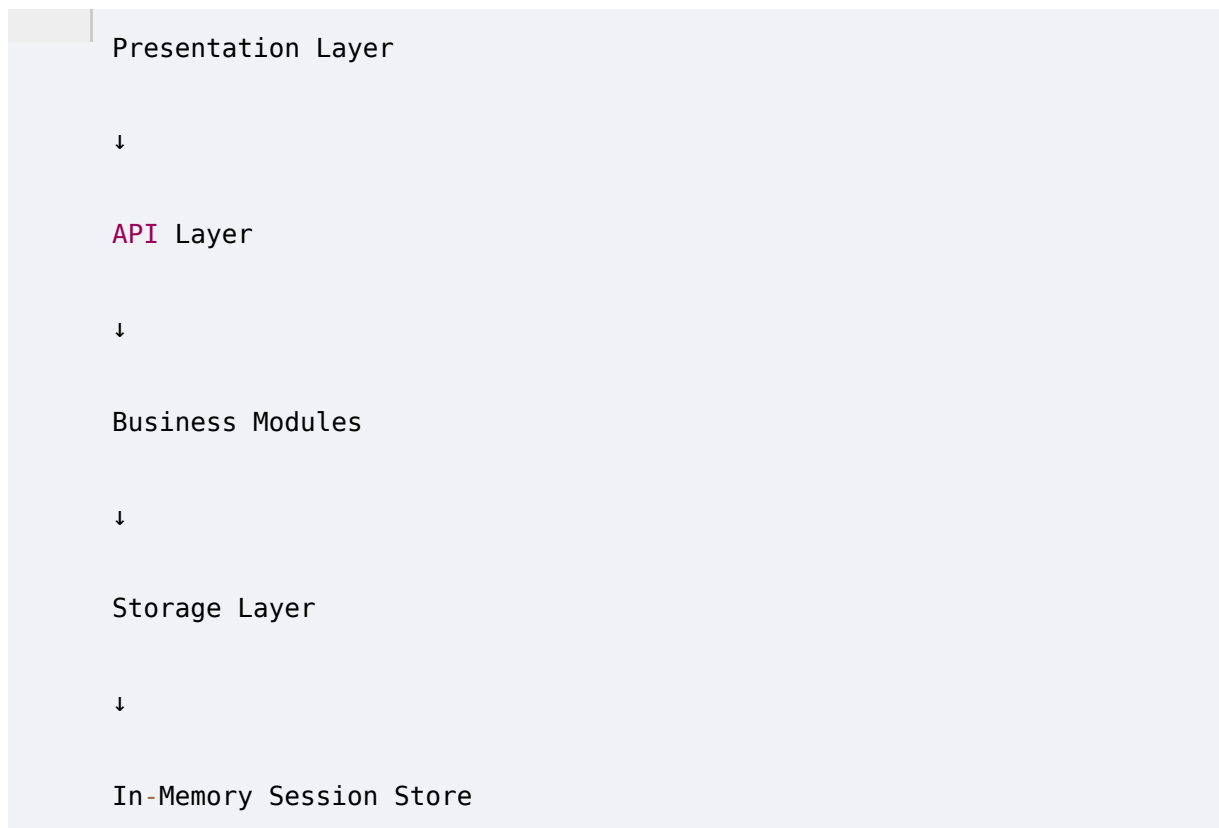




3. System Architecture

Architecture Pattern

Layered Modular Architecture



Each layer has only one responsibility.

4. Storage Architecture

The MVP does **not** use persistent storage.

Everything exists only during the active session.

```
sessions = {  
  
    session_id: SessionState  
  
}
```

SessionState

```
SessionState  
  
dataset  
  
analytics  
  
prediction  
  
insights  
  
decision  
  
report
```

Every uploaded dataset creates one isolated session.

Storage Responsibilities

storage.py exposes only:

```
create_session()  
  
get_session()  
  
save_dataset()  
  
save_analytics()  
  
save_prediction()  
  
save_insights()  
  
save_decision()
```

```
save_report()
```

```
clear_session()
```

Modules never access the storage dictionary directly.

5. Project Structure

```
backend/
```

```
|
```

```
|— main.py
```

```
|— models.py
```

```
|— schemas.py
```

```
|— storage.py
```

```
|— config.py
```

```
|— utils/
```

```
|
```

6. Backend Technology Stack

Framework

- FastAPI

Data Processing

- Pandas
- NumPy

Machine Learning

- Scikit-learn

Validation

- Pydantic

Reporting

- ReportLab

LLM Integration

- Provider Agnostic

Utilities

- Python Standard Library
-

7. Frontend Technology Stack

Framework

- React

Language

- TypeScript

Styling

- Tailwind CSS

UI Components

- shadcn/ui

Charts

- Recharts

HTTP

- Axios
-

8. Canonical Domain Models

These are the only objects exchanged between modules.

Dataset

```
Dataset
```

```
id: str
```

```
filename: str
```

```
rows: int
```

```
columns: int
```

```
schema: list

quality: QualityReport

cleaned_dataframe: DataFrame
```

Analytics

```
Analytics

summary

correlations

charts

outliers
```

Prediction

```
Prediction

problem_type

metrics

feature_importance

predictions
```

Insight

```
Insight

id

title

description

severity
```

category

evidence

confidence

Decision

Decision

scenario_name

expected_impact

confidence

assumptions

recommendation

counter_analysis

Report

Report

sections

generated_at

pdf_path

9. Module Specifications

Dataset Module

Responsibility

Validate and clean uploaded datasets.

Input

CSV File

Output

Dataset

Responsibilities

- Read CSV
 - Detect data types
 - Handle missing values
 - Remove duplicates
 - Generate quality score
 - Store cleaned DataFrame
-

Analytics Engine

Input

Dataset

Output

Analytics

Responsibilities

- Summary statistics
 - Correlations
 - Outlier detection
 - Distribution analysis
 - Chart specifications
-

Prediction Engine

Input

Dataset

Target Column

Output

Prediction

Responsibilities

- Detect ML problem
 - Train model
 - Evaluate model
 - Generate predictions
 - Feature importance
-

Insight Engine

Input

Analytics

Prediction

Output

List

Responsibilities

- Generate business findings
 - Categorize insights
 - Assign severity
 - Attach evidence
 - Estimate confidence
-

Decision Simulator

Input

Analytics

Prediction

Scenario

Output

Decision

Responsibilities

- Estimate business impact
 - Generate assumptions
 - Produce recommendation
 - Generate counter analysis
-

Executive Report

Input

Dataset

Analytics

Prediction

Insights

Decision

Output

PDF

Sections

Dataset Summary
Analytics Summary
Key Findings
Recommendations
Counter Analysis
Conclusion

Executive Advisor

Input

Insight

or

Chart

Output

AdvisorResponse

	AdvisorResponse
	explanation

Purpose

Explain charts and insights in plain business language.

10. API Specifications

Upload Dataset

	POST
	/dataset/upload

Returns

Dataset

Analytics

	GET
--	-----

```
/dataset/{id}/analytics
```

Returns

Analytics

Prediction

```
POST
```

```
/dataset/{id}/predict
```

Body

```
{  
  "target_column": "Sales"  
}
```

Returns

Prediction

Insights

```
GET
```

```
/dataset/{id}/insights
```

Returns

List

Decision

```
POST
```

```
/dataset/{id}/decision
```

Body

```
{  
  "scenario_name": "Price Increase",  
  "input": 10  
}
```

Returns

Decision

Report

GET

/dataset/{id}/report.pdf

Returns

PDF

Executive Advisor

POST

/dataset/{id}/advisor

Body

```
{  
  "chart_id": "chart_2"  
}
```

Returns

```
{  
  "explanation": "..."  
}
```

11. Frontend Pages

Overview

Displays

- Dataset Summary
 - Quality Score
 - KPI Cards
 - Alerts
-

Analytics

Displays

- Charts
 - Correlations
 - Statistics
-

Predictions

Displays

- Metrics
 - Predictions
 - Feature Importance
-

Report

Displays

- Report Preview
 - PDF Download
-

Decision Center

Displays

- Scenario Input
 - Recommendation
 - Counter Analysis
 - Assumptions
 - Confidence
-

12. Engineering Principles

Rule 1

Single Responsibility Principle.

Each module performs one task only.

Rule 2

Structured Objects Only.

Modules exchange canonical objects.

Never dictionaries.

Rule 3

One Source of Truth.

Every calculation is performed once.

Dashboard, Report, Advisor, and Decision Simulator render the same underlying objects.

Rule 4

Presentation Layer Never Calculates.

Frontend only renders.

Backend performs all processing.

Rule 5

LLM Isolation.

LLM never accesses raw CSV.

It receives only:

- Analytics
- Prediction
- Insight
- Decision

objects.

Rule 6

No Cross-Module Dependencies.

Example

Prediction Module cannot import Report Module.

Report Module cannot import Analytics internals.

Communication occurs only through domain objects.

Rule 7

Loose Coupling.

Every module should be independently testable.

13. Error Handling

Each module returns standardized errors.

```
ErrorResponse
```

```
status
```

```
code
```

```
message
```

```
details
```

Examples

- Invalid CSV
- Empty Dataset
- Unsupported Data Type
- Prediction Failed
- Report Generation Failed

14. Logging Strategy

Every module logs:

- Start
- Completion
- Processing Time
- Errors

Log Levels

- INFO
- WARNING
- ERROR

15. Testing Strategy

Each module is verified independently before integration.

Dataset

- CSV parsing
- Missing values

- Duplicates

Analytics

- Summary correctness
- Correlations
- Charts

Prediction

- Model selection
- Metrics

Insight

- Output schema
- Confidence
- Categories

Decision

- Recommendation
- Counter Analysis

Report

- PDF generation

Advisor

- Explanation quality
-

16. Performance Targets

Metric	Target
CSV Upload	< 5 sec
Analytics Generation	< 8 sec
Prediction	< 15 sec
Insight Generation	< 10 sec

Report Generatio n	< 5 sec
Decision Simulation	< 8 sec
Chart Explanatio n	< 5 sec

Target dataset size

- Up to **100,000 rows** (assuming a typical business dataset and sufficient local memory)

17. Security (MVP)

Since there is:

- No authentication
- No persistence
- No user accounts

Security scope is limited to:

- File type validation
- File size limits
- Session isolation
- Input validation
- Temporary in-memory storage

18. Build Order

Phase 1

- Storage
- Models
- Schemas

Phase 2

- Dataset Module
-

Phase 3

- Analytics Engine
-

Phase 4

- Prediction Engine
-

Phase 5

- Insight Engine
-

Phase 6

- Decision Simulator
-

Phase 7

- Executive Report
-

Phase 8

- Executive Advisor
-

Phase 9

- Frontend Integration
-

Phase 10

- End-to-End Testing
 - Bug Fixes
 - Demo Preparation
-

19. Acceptance Criteria

A build is considered complete only if:

- ☒ CSV upload succeeds.
- ☒ Dataset quality is calculated.
- ☒ Analytics are generated.
- ☒ Prediction runs successfully.
- ☒ Insights are produced as structured objects.
- ☒ Decision simulation completes.
- ☒ Executive Report generates successfully.
- ☒ Executive Advisor explains charts or insights.

-  All pages consume the same canonical objects.
 -  Full pipeline executes without manual intervention.
-

20. Future Technical Roadmap

Beyond the internship MVP:

- PostgreSQL persistence
 - Authentication & user sessions
 - Async background jobs
 - Dataset versioning
 - Forecasting engine
 - Plugin architecture for analytics modules
 - Additional decision scenarios
 - Multi-tenant support
 - Report templating
 - External data connectors
 - Deployment with Docker & CI/CD
-

21. Final Technical Principles (Locked)

1. **Modular by Design** – Each module has one clear responsibility.
2. **Object-First Architecture** – Canonical Pydantic models are the only data exchanged between modules.
3. **One Source of Truth** – Business calculations occur once and are reused everywhere.
4. **Backend-Driven Intelligence** – All analytics, predictions, and reasoning are produced in the backend; the frontend only presents results.
5. **Session-Based Simplicity** – The MVP uses isolated in-memory sessions to maximize development speed and minimize complexity.
6. **Fresh, Maintainable Codebase** – Every component is built from scratch with clean interfaces, making future migration to persistent storage straightforward.